

Comparative Analysis of Principal Component Analysis and Support Vector Machines Kernels for American Sign Language Recognition

Le Tien Dat, Nguyen Thu Uyen, Nguyen Gia Khanh

Abstract—Sign language is a necessary in communication with people with disabilities in hearing. This paper explores and attempts to address the task by comparing four approaches of Support Vector Machines (SVM) to detect the American Sign Language (ASL) alphabet: by using Radial Basis Function (RBF) Kernel and Polynomial Kernel, in which each method is experimented both separately and paired with Principal Component Analysis (PCA). We use the MNIST Sign Language Dataset with 27,455 training samples and 7172 test samples, on which four models will be trained and evaluated in terms of accuracy and running time. The results indicate that pairing with PCA improves efficiency in time complexity by reducing noise, and together with SVM and RBF/Polynomial Kernels, yield promising accuracy.

Index Terms—Support Vector Machines, Radial Basis Function Kernel, Polynomial Kernel, Principal Component Analysis, Sign Language Recognition

1 INTRODUCTION

SIGN languages are languages that use non-verbal gestures to convey meaning, instead of spoken words. They are crucial for people with hearing or speech impairment as a useful means of communication. In contrast, only a small population of hearing people is able to use sign language, thus isolating the deaf-mute community. Most communication technologies are developed to support spoken and written language, yet exclude sign language. The sign language dialects also contribute to the difficulty, since they vary across regions, ages, and races. Thus, many deaf-mute people may find difficulties in using these technologies. The need for easy and accessible deaf-friendly devices made us believe that the ability to translate sign language into verbal language at low cost should enable people with hearing impairment to commute and actively participate in the workforce.

We solve the above problem by utilizing the help of Machine Learning. Due to the robust performance of Convolutional Neural Networks (CNN) [1] in recognition tasks, classical machine learning algorithms such as SVM tend to be overlooked. However, similar work was conducted by researchers [2], whereas SVM was compared to K-Nearest Neighbors (KNN) in sign language detection, showing positive results. Our paper, on the other hand, emphasizes the capability of PCA and SVM Kernels in classification through a comparative analysis, and expects to determine the most relatively accurate solution.

This paper aims to recognize the ASL alphabet by extracting characteristics and classifying the Sign Language MNIST dataset. Initially, we apply the PCA method to project high-dimensional data to a lower-dimensional space. Subsequently, we use SVM with RBF and Polynomial kernel strategies for classification.

We successfully use PCA and SVM to classify ASL hand

gestures. The best result shows 85.75% accuracy on the PCA&SVM model using Polynomial kernel, while the worst result 78.21% in Polynomial model without PCA.

1.1 Contribution:

- 1) The experiment was conducted on four models consisting RBF Kernel and Polynomial, in which we tried including PCA, and without PCA.
- 2) We conclude that for small dataset, SVM still work with reasonable results and benefit from the simpler implementation and computational cost.
- 3) Our experiment shows that small models with PCA perform better and faster than the others without it. The Polynomial models benefit the most, and overall perform the best out of the four models, which indicates that the Polynomial Kernel is more sensitive to preprocessing data using PCA than the RBF kernel.

2 RELATED WORK

Sign Language Recognition is a complex research area in computer vision. In recent years, many studies have been carried out to research this field, notably in machine learning approaches.

Pigou, Lionel et al. suggested using CNNs for sign language recognition in their 2014 study [1]. They were able to recognize 20 Italian gestures with high training accuracy using a dataset of 4800 images. However, their approach requires a lot of computational resources and large labeled datasets for training, making it costly and inaccessible for many amateur researchers.

Another work by Poladiya, Parth et al [2], compared SVM and KNN at all Pyramidal Histogram of Gradient (PHOG) levels. Their results showed that SVM outperformed KNN

with 81% accuracy compared to KNN's 77%. However, the paper did not explore combining pre-processing strategies, nor experiment with kernel methods to increase accuracy.

Another proposed approach is to use PCA for feature extraction and SVM for classification. In 2018, Hai and Thinh proposed an automatic feature extraction method for Vietnamese Sign Language in real-life, which has an accuracy of 93.3% [8]. Another 2019 study by Lwin and Tun [7] for Myanmar sign language recognition resulted in 89% accuracy. This method was also used by Novianty and Azmi for Bahasa Indonesia SL in 2021, with 76.8% accuracy [6]. However, they did not mention or explore complex kernel strategies for SVM. It is suggested that implementing more advanced kernel methods, such as RBF and Polynomial kernels, can further improve the performance.

Our aim is to increase the accuracy in predicting the ASL alphabet and compare our methods, using SVM as a classifier using RBF and Polynomials kernel strategies, pairing with PCA.

3 BACKGROUND

In this section, we discuss the basic concepts of Principal Component Analysis, Support Vector Machines, kernel strategies, and why we choose the American Sign Language.

3.1 American Sign Language (ASL)

The American Sign Language ranked 7th in population, making it the most widely used SL in the world and the fourth most used linguistic in North America. More than 20 other nations in the Caribbean, Southeast Asia, and Africa, where English is the primary language, use ASL to communicate with deaf-mute people. [12]. The ASL datasets are well-documented and widely used. Its alphabet is easy and concise, containing only the letters with no diacritics or two-handed gestures. Due to its popularity, we chose the ASL to experiment on with our SVM models.

3.2 Principal Component Analysis (PCA)

Principal Component Analysis, or PCA is a dimensionality reduction technique that compresses multivariate data into a smaller subspace while retaining important information of the original dataset. Prior to training with PCA, data will be preprocessed by normalizing each feature to have a mean of 0 and variance of 1.

To reduce feature dimension, PCA performs linear transformation via some orthonormal basis $[w^{(1)}, \dots, w^{(l)}]$ on the original vectors to obtain compressed vectors $t^{(i)}$'s:

$$t^{(i)} = [(x^{(i)})^T w^{(1)}, \dots, (x^{(i)})^T w^{(l)}] \in \mathbb{R}^l$$

Such basis is chosen so that the variance of each new feature is maximized. With Lagrange Multipliers, it can be shown that $w^{(i)}$ is the eigenvector of Σ that corresponds to the maximal eigenvalue, whereas Σ is the covariance matrix of our original dataset.

Thus the l principal components correspond with the top l eigenvectors of Σ

3.3 Support Vector Machines

Support Vector Machines in classification are models that classify data according to their margins, developed based on statistical learning frameworks of VC theory proposed by Vapnik (1982, 1995) and Chervonenkis (1974). SVMs are widely used to solve binary classification tasks. However, multiclass classification is also feasible by applying binary problem multiple times.

a) Hard-margin and the optimization problem

In the context of Hard-margin classifier, we suppose the dataset is linearly separable. This means that there exists $w \in \mathbb{R}^p, b \in \mathbb{R}$ such that for all training examples $(x^{(i)}, y^{(i)})$:

$$y^{(i)}(w^T x^{(i)} + b) \geq 0$$

To further formalize the concept of Hard-margin SVM, for each training example, we introduce their margins:

Functional Margin: $\hat{\gamma}^{(i)} = y^{(i)}(w^T x^{(i)} + b), \forall i = 1, \dots, n$. We also define the functional margin of a training set as: $\hat{\gamma} = \min_{i=1, \dots, n} \hat{\gamma}^{(i)}$.

Geometric Margin: $\gamma^{(i)} = \frac{\hat{\gamma}^{(i)}}{\|w\|}$. This quantity represents the distance from $x^{(i)}$ to the hyperplane $w^T x + b = 0$ in p -dimensional space times $y^{(i)} \in \{-1, 1\}$. We also define the geometric margin $\gamma = \min_{i=1, \dots, n} \gamma^{(i)}$.

In Hard-margin SVM, we separate the training set with some hyperplane $w^T x + b = 0$.

Afterwards, prediction of the unlabeled test sample x is classified in the following manner:

$$x = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

By purposely scaling w, b , the optimization problem for Hard-margin SVM, aiming to maximize the geometric margin is derived:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad (1)$$

$$s.t : y^{(i)}(w^T x^{(i)} + b) \geq 1, \forall i = 1, \dots, n \quad (2)$$

This a Quadratic Programming problem, which can be solved using libraries such as CVXOPT or QPLIB.

b) Lagrange Duality - SVM's Dual Problem

Lagrange Duality of a convex optimization problem (also called Primal problem) is a problem that is equivalent to Primal problem under certain conditions. Within context of SVM, it helps introducing the Kernels as well as being more efficient to solve for in many cases.

A Primal optimization problem is stated as:

$$\min_{w \in \mathbb{R}^p} f(w) \quad (3)$$

$$s.t : g_i(w) \leq 0, i = 1, \dots, k \quad (4)$$

$$h_i(w) = 0, i = 1, \dots, l \quad (5)$$

where (4), (5) are called *Primal constraints*.

The corresponding Dual problem is:

$$\max_{\alpha, \beta} \theta_D(\alpha, \beta) \quad (6)$$

$$s.t : -\alpha_i \leq 0, \forall i = 1, \dots, k \quad (7)$$

By *Slater's Theorem*, SVM meets the sufficient conditions so that its Primal and Dual problem are equivalent. Which means for optimal points w^*, α^*, β^* , we must have $f(w^*) = \theta_D(\alpha^*, \beta^*)$. Also, these values satisfy the *Karush-Kuhn-Tucker* conditions:

$$\frac{\partial}{\partial w_i} \mathcal{L}(w^*, \alpha^*, \beta^*) = 0, \forall i = 1, \dots, p = \dim(w) \quad (8)$$

$$\frac{\partial}{\partial \beta_i} \mathcal{L}(w^*, \alpha^*, \beta^*) = 0, \forall i = 1, \dots, l \quad (9)$$

$$\alpha_i^* g_i(w^*) = 0 \quad (10)$$

$$g_i(w^*) \leq 0 \quad (11)$$

$$\alpha_i^* \geq 0 \quad (12)$$

We can solve for the Dual problem of SVM:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \quad (13)$$

$$\alpha_i \geq 0, i = 1, \dots, n \quad (14)$$

$$\sum_{i=1}^n \alpha_i y^{(i)} = 0 \quad (15)$$

The relationship of α^* and w^*, b^* is derived from Karush-Kuhn-Tucker conditions:

$$w^* = \sum_{i=1}^n \alpha_i^* y^{(i)} x^{(i)}$$

$$b^* = - \frac{\max_{i: y^{(i)} = -1} (w^*)^T x^{(i)} + \min_{i: y^{(i)} = 1} (w^*)^T x^{(i)}}{2}$$

c) Soft-margin SVM

For the case of Soft-margin, which works for non-linearly separable dataset, a regularization term is added:

$$\min_{\xi, w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (16)$$

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, i = 1, \dots, n \quad (17)$$

$$\xi_i \geq 0, i = 1, \dots, n \quad (18)$$

By recomputing the Lagrangian, the Dual problem with regularization is:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \quad (19)$$

$$0 \leq \alpha_i \leq C, i = 1, \dots, n \quad (20)$$

$$\sum_{i=1}^n \alpha_i y^{(i)} = 0 \quad (21)$$

This Dual problem manages to get rid of the complex term $C \sum_{i=1}^n \xi_i$ that affects the efficiency of QP libraries, slowing the training and prediction.

3.4 Kernels

a) Feature Mapping

Consider a dataset with n examples $(x^{(i)}, y^{(i)})$ where

$$x^{(i)} \in \mathbb{R}^p$$

A Feature Mapping is a function:

$$\begin{aligned} \phi : \mathbb{R}^p &\rightarrow \mathbb{R}^d \\ x &\rightarrow \phi(x) \end{aligned}$$

In general, Feature Mapping creates more features with the intention of adding new interesting information for training.

b) Kernel Trick in SVMs

Kernel Trick is a technique both leverages the input dimension and retain the computational complexity. By changing (mostly increasing) the dimension, our initially inseparable data could be mapped into a space in which it now can be divided by a hyperplane.

Within the context of SVM, Kernel Trick uses the concept of Kernel functions to precompute the inner product values.

c) Kernel functions

To use the Kernel Trick, we define a function $K : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ to be a Kernel Function if there exists a feature mapping $\phi : \mathbb{R}^p \rightarrow \mathbb{R}$ such that:

$$K(x, z) = \langle \phi(x), \phi(z) \rangle = \phi(x)^T \phi(z), \forall x, z \in \mathbb{R}^p$$

For example, the function $K(x, z) = 1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3$ is a kernel function. Despite the dimension of ϕ can be really large, computing $\langle \phi(x), \phi(z) \rangle$ using the kernel function above is significantly efficient.

In Machine Learning tasks, there are common Kernels such as:

Linear:

$$K(x, z) = x^T z$$

Polynomial:

$$K(x, z) = (\gamma x^T z + r)^d$$

Radial Basis Function:

$$K(x, z) = \exp \left(- \frac{\|x - z\|^2}{2\gamma^2} \right)$$

Sigmoid:

$$K(x, z) = \tanh(ax^T z + c)$$

In the scope of this paper, we focus on Polynomial and RBF. Further discussion on the efficiency of these two with respect to the task of Sign Language Classification will be analyzed in the experiments.

4 METHODOLOGY

In this section, we propose our approach to comparatively analyze the performance of classical machine learning models in recognizing and classifying the American Sign Language (ASL) gestures.

4.1 Research Questions

RQ1. Which variant is the most suitable for recognizing ASL gestures?

RQ2. Does PCA enhance speed and accuracy?

RQ3. Are those four models suitable for Sign Language recognition?

Our ultimate goal is to analyze the speed and accuracy of these approaches to see how classical Machine Learning algorithms perform on a sign language dataset.

4.2 Sign Language MNIST Dataset

For this study, we use the MNIST Sign Language dataset. This dataset is a grayscale image collection of 26 letters in ASL alphabet. This dataset excludes "J" and "Z", since they require motion gestures. Our dataset consists of two portions: the Train set containing 27,455 samples, and the Test set with 7172 samples. Figure 1 shows 6 hand gesture samples from the MNIST dataset. Each of them and the rest of the samples has a resolution of 28x28 pixels, varying from 0 (Black) to 255 (White). The labels from A – Y (J, Z are absent) are distributed evenly in the Train set, each letter has approximately 1000 occurrences.

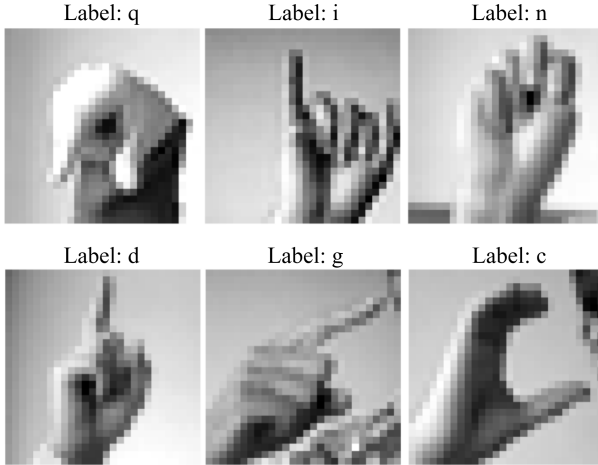


Fig. 1: MNIST Sign Language Samples

4.3 Preprocessing

Firstly, the data experiences the preprocessing phase, which consists of sampling and data normalizing.

Table ?? gives us an overview of the numbers of samples for each letter in the alphabet. The distribution is relatively balanced, ranging from 957 samples (letter E) to 1294 (letter R). Despite this, randomly sampling can cause uneven label frequency which potentially worsen the performance. As a result, stratified sampling technique is applied to overcome this difficulty.

The training set is then split into an 80:20 ratio using stratified sampling with respect to the labels to obtain a new training set X_{train}, Y_{train} and a validation set X_{valid}, Y_{valid} . These datasets are utilized for model evaluation and hyperparameter tuning. Additionally, the pixel values are normalized to the interval [0,1] by dividing

each intensity value by 255 for better performance. It also aids algorithms that rely on distance metrics, which is SVM in this case.

Letter	Samples	Letter	Samples
A	1126	B	1010
C	1144	D	1196
E	957	F	1204
G	1090	H	1013
I	1162	K	1114
L	1241	M	1055

Letter	Samples	Letter	Samples
N	1151	O	1196
P	1088	Q	1279
R	1294	S	1199
T	1186	U	1161
V	1082	W	1225
X	1164	Y	1118

TABLE 1: Sample counts for different letter indices.

4.4 Fine-Tuning

a) Hyperparameters

In order for the models to perform with their full potential, tuning hyperparameters is a prerequisite. For SVM, the regularization coefficient C controls the trade-off between training and testing error. RBF and Polynomial kernels, on the other hand, have their own parameters:

For RBF: A single training example has the influence implied as γ in the kernel function:

$$K(x, z) = \exp \left(- \frac{\|x - z\|^2}{2\gamma^2} \right)$$

For Polynomial: The free coefficient r will be set to 0 as default. The other ones are coefficient γ , and degree d in

$$K(x, z) = (\gamma x^T z + r)^d$$

b) Fitting PCA

We train PCA after the data is rescaled. Note that it should be fitted on the training dataset X_{train} only, as we only determine a unique subspace to project high-dimensional data onto. When evaluating or predicting a set of samples X , it should be preprocessed appropriately like stated in the previous part, then transformed with our fitted PCA before feeding into SVM. The number of principal components $n < 784$ of PCA will be determined during fine-tuning.

c) Prepare a Smaller Batch

When fine-tuning, training is done repetitively. This leads to huge computational complexity, even if PCA already reduced the dimension. To tackle this, we extract a mini batch X_{mini_train} from X_{train} with 25% proportion by stratified sampling. We then conduct tuning with X_{mini_train} .

d) Gridsearching

Finally, the hyperparameters will be chosen by Gridsearch - a method for brute-force training on a list of parameters.

Additionally, we compute the accuracy when training the model with each combination of parameters on X_{valid} . Now with both training accuracy obtained from Gridsearch and the computed validation accuracy, we deduce the combination that performs the best for ultimate testing.

4.5 Experimental Setup

The models are implemented in Python, using libraries such as NumPy, Pandas, and Scikit-learn. NumPy is a library which helps shorten the work done by performing mathematical operations and functions on arrays or matrices of numbers, making it ideal for scientific computing tasks. Pandas supports different types of relational operations such as joins, merging, etc. Moreover, it provides a fast and easy way to load data from multiple sources, such as CSV files or SQL databases. Scikit-learn is utilized in this model in order to split the test set and validation set more easily. All the implementations and evaluations are conducted on:

- **Device:** Acer Nitro 5 Laptop - Windows 11
- **Processor:** 12th Gen Intel(R) Core(TM) i5-12500H 3.10 GHz
- **RAM:** 16.0 GB
- **GPU:** NVIDIA Geforce RTX 3050

4.6 Metrics

We use classification accuracy and confusion matrix as main metrics. Training and test time are tracked for performance evaluation, and confusion matrices provide information about patterns of misclassifications among classes of letters.

5 EXPERIMENTAL RESULTS

5.1 Fine-Tuning Result

The Grid-searching process is conducted on X_{mini_train} with 5491 examples for each of the four aforementioned models. Each model is cross-validated 5 times during the process to obtain the mean training accuracy. Additionally, each combination of parameter is used to obtain a validation score on X_{valid} .

Below are parameter grids of our models on which we seek the most prominent combinations in terms of validation and training score, paired with their corresponding tuning result. The result tables consist of 5 combinations that yielded the best validation score on X_{valid} .

C	0.1, 1, 10, 100
γ	0.001, 0.01, 0.1, 1

TABLE 2: Potential parameters in RBF Kernel

C	γ	Training Accuracy	Validation Accuracy
100.0	0.010	0.9953	0.9998
10.0	0.010	0.9898	0.9956
100.0	0.001	0.9856	0.9918
10.0	0.100	0.9756	0.9867
100.0	0.100	0.9756	0.9867

TABLE 3: Fine-Tuning results for RBF Kernel

Number of components (n_p)	20, 30, 50, 100
C	0.1, 1, 10, 100
γ	0.001, 0.01, 0.1, 1

TABLE 4: Potential parameters in RBF Kernel with PCA

n_p	C	γ	Training Accuracy	Validation Accuracy
100	100.0	0.010	0.9947	0.9993
50	100.0	0.010	0.9918	0.9982
30	100.0	0.010	0.9845	0.9965
100	10.0	0.010	0.9798	0.9900
100	100.0	0.100	0.9758	0.9862

TABLE 5: Fine-Tuning results for RBF Kernel with PCA

C	0.1, 1, 10, 100
Degree (d)	2, 3, 4, 5

TABLE 6: Potential parameters in Polynomial Kernel

C	d	Training Accuracy	Validation Accuracy
1.0	3	0.9938	0.9991
10.0	3	0.9938	0.9991
100.0	3	0.9938	0.9991
0.1	3	0.9936	0.9989
1.0	2	0.9951	0.9982

TABLE 7: Fine-Tuning results for Polynomial Kernel

Number of components (n_p)	20, 30, 50, 100
C	0.1, 1, 10, 100
Degree (d)	2, 3, 4, 5

TABLE 8: Potential parameters in Polynomial Kernel with PCA

n_p	C	d	Training Accuracy	Validation Accuracy
100	10.0	2	0.9852	0.9973
50	100.0	2	0.9860	0.9971
100	100.0	2	0.9874	0.9967
30	100.0	2	0.9856	0.9962
50	10.0	2	0.9822	0.9944

TABLE 9: Fine-Tuning results for Polynomial Kernel with PCA

5.2 Test Result

Finally, we merge X_{train} and X_{valid} back into the original training set to perform testing. The accuracies obtained on a completely unseen X_{test} results are shown below:

RBF Kernel: For $C = 100, \gamma = 0.01$ we obtain **0.822783** accuracy in test results.

RBF Kernel with PCA: For $C = 100, \gamma = 0.01, n_p = 100$ we obtain **0.840769** accuracy in test results.

Polynomial Kernel: For $C = 100, d = 3$ we obtain **0.782069** accuracy in test results.

Polynomial Kernel with PCA: For $C = 10, d = 2, n_p = 100$ we obtain **0.857501** accuracy in test results.

Specific predictions of labels are represented using confusion matrices:

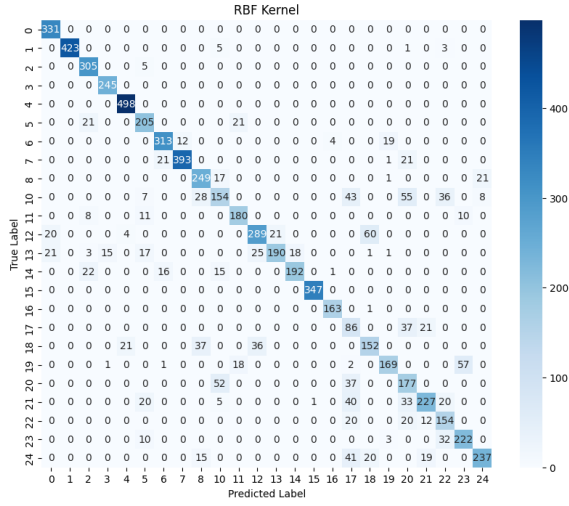


Fig. 2: RBF Kernel

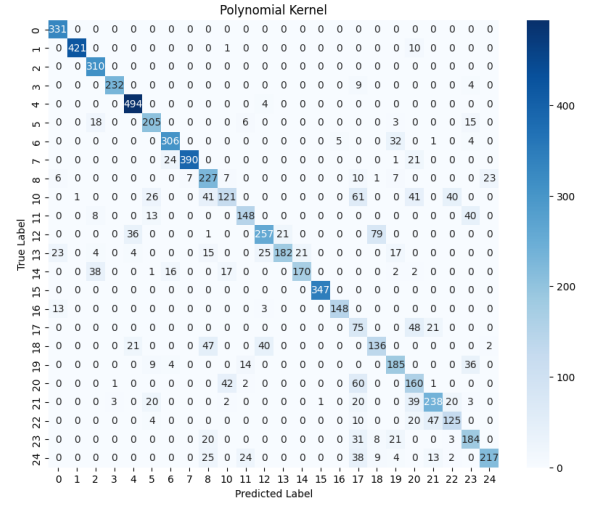


Fig. 4: Polynomial Kernel

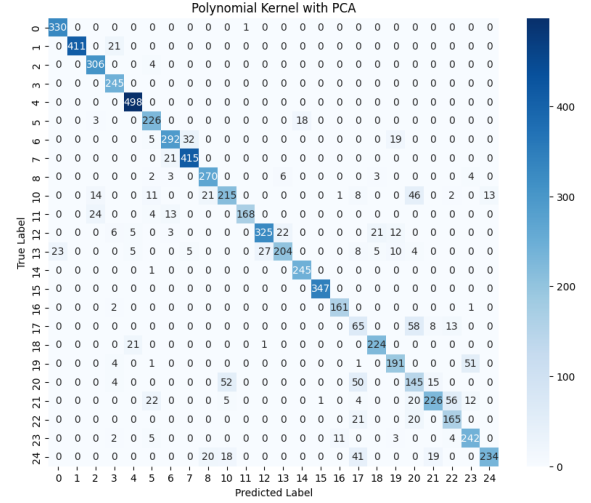


Fig. 5: Polynomial Kernel with PCA

Training and validating speed of the models are enlisted below. For the ones that applied PCA, the speed of fitting PCA is not taken into account (seconds).

Model	Training Speed(s)	Testing Speed(s)
RBF Kernel	20.6	14.1
RBF Kernel + PCA	3.9	2.3
Poly Kernel	13.1	1.8
Poly Kernel + PCA	4.7	0.7

TABLE 10: Training and Testing Speed

From the confusion matrices, we can summarize and round the accuracy per label of each model as followed:

Attributes (%)	RBF Kernel	RBF Kernel + PCA
total acc	82.28	84.08
std	15.69	12.99
min	46.53	60.98
max	100	100

TABLE 11: RBF Kernel Label Accuracy Analysis

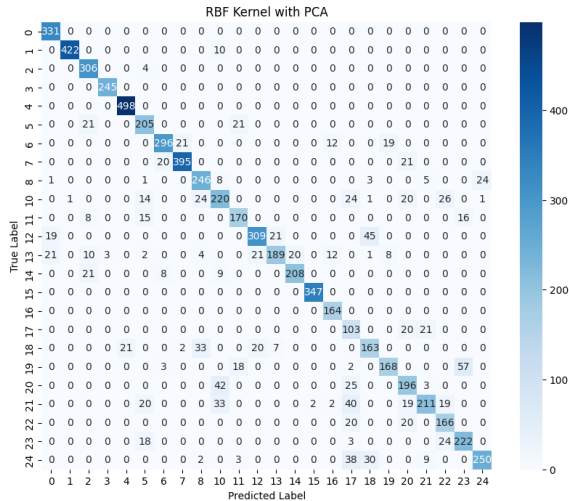


Fig. 3: RBF Kernel with PCA

Attributes (%)	Poly Kernel	Poly Kernel + PCA
total acc	78.21	85.75
std	17.80	15.37
min	36.56	53.47
max	100	100

TABLE 12: Polynomial Kernel Label Accuracy Analysis

6 DISCUSSION

In this paper, we aim to experiment with the impact of PCA and kernel functions on SVM classification for ASL recognition. Our findings indicate that PCA generally improves the accuracy of SVM models, as seen in both RBF and Polynomial kernels.

6.1 Fine-Tuning Result

The parameter ranges were chosen heuristically based on the attributes of the problem such as dataset size and complexity of images. For RBF Kernel, coefficient $C = 100$ yields the highest accuracy on both train and valid sets. In the scope of SVM, lower value of C tends to strengthen the regularization, thus with the moderate complexity of our dataset, such small C like $C = 0.1$ or $C = 1$ give lower results (84.2% train accuracy and 87.5% validation accuracy with $C = 1, \gamma = 0.01$ without PCA confirmed during experiment).

Additionally, the number of principal components being 100 performs most accurately in both Kernels, it balances well between eliminating redundant noise and retaining crucial information. Aside from C , the coefficient $\gamma = 0.01$ appears at the top in RBF Kernel result chart, and $d = 3, 2$ work best for Polynomial Kernel with and without PCA respectively. Based on the conclusions above, we choose $C = 100, \gamma = 0.01$ for ultimate testing with RBF Kernel, and the same combination is applied for RBF Kernel and PCA with $n_p = 100$. With Polynomial Kernel and PCA, $n_p = 100, C = 10$, and $d = 2$ will be evaluated on the test set.

However, fine-tuning on Polynomial Kernel without PCA yielded three identically best combinations. By recalling the role of C , since the dataset is relatively simple, strong regularization is not compulsory. Therefore $C = 100, d = 3$ was chosen for final testing.

6.2 General Result

a) Testing Speed

Table 10 showed that PCA contributes to reducing training and testing speed by lowering the dimension of input data. The RBF without PCA model's training takes 20.6s, and the testing speed is 14.1s whereas with PCA, it reduces these times to just 3.9s and 2.3s, respectively. Moreover, the Polynomial model with PCA also showed a better speed result with 4.7s for training and 0.7s for testing, compared to 13.1s and 1.8s without PCA. This not only indicates that models with PCA preprocessing consistently show faster results, but also suggests that Polynomial kernels are more suitable for the ASL recognition tasks in real-time.

b) Classification Accuracy

Regarding recognition accuracy, the RBF SVM model alone resulted in an accuracy of 82.28%, while combining with PCA improved the result to 84.08%. On the other hand, the Polynomial SVM achieved 78.21% accuracy, whereas model resulted in 85.75% when applied PCA. This proves PCA's ability to reduce noise and redundant features. SVM with RBF kernel had slight improvement, which shows that non-linear mapping is less sensitive to dimensionality reduction. In contrast, the Polynomial mapping improvement is more remarkable.

6.3 Result Across Labels

In addition, according to Table 11 and Table 12, the standard deviation of accuracy across different classes is lower when PCA is applied, suggesting that PCA helps stabilize the classifier's performance across different signs. The lower standard deviation in RBF when compared with Polynomial also indicates that the accuracy across different labels is more stable in the RBF model, while the Polynomial models behave inconsistently.

Furthermore, PCA was able to increase the minimal accuracy on both Kernels, with a considerable 14% growth on RBF and 17% on Polynomial. This means the models are capable of detecting tricky letters better with the aid of PCA.

Additionally, the confusion matrices shown in Figure 2, 3, 4, 5 suggest interesting insights about the classification accuracy of each model. Some letters are precisely predicted by all the models, as shown by the strong diagonal values: 0, 1, 5, and 15. This implies their features are well captured by the models. In contrast, other letters experience misclassification, particularly the ones labeled 10, 12, 19, and 22 according to the matrices. In the Polynomial Kernel without PCA, label 10 is often confused with label 12, and label 19 is misclassified into multiple other categories. Applying PCA generally improves the classification of some difficult letters, specifically labels like 10 and 12, as seen in the decreased misclassified letters in both kernels. However, there are still a few letters that slightly remain less efficient, even after PCA is applied, such as label 22.

6.4 Related Research

In Wang's study [9] on facial recognition using the Cambridge ORL Face database, their PCA and SVM model consistently outperforms SVM, with the highest accuracy of 94.29%. We can see a similar trend in our experiments, although the accuracy improvements in our ASL recognition task are less impactful. Moreover, Ramesh et al. study (2021) showed that preprocessing using PCA improved the performance of SVM in handwritten Kannada digit recognition datasets, resulting in 95.44% accuracy. [10]. Although there are variations of absolute performance metrics due to their own characteristics of the domain and datasets, the trend is unchanged: PCA contributes to reducing redundant information and alleviating noise, which helps in the construction of more effective decision boundaries. This confirms our finding that PCA contributed

to reducing noise and enhancing accuracy across various datasets.

Regarding deep learning algorithms for sign language recognition, the CNN approach shown in [1] achieved accuracies of up to 90%, surpassing the highest accuracy of 85.75% observed in our experiments. However, the high accuracy comes with a cost of increasing complexity and training times. In contrast, the SVM with PCA offered an alternative that is suitable for limited resources, strict runtime requirements scenarios and smaller datasets, while CNNs and ANNs have higher chances of overfitting.

6.5 Limitations and Future work

While this study shows promising results in using PCA and SVM for sign language detection, there exist some limitations that need improvements. Firstly, dynamic gestures such as "J" and "Z" have been excluded from the dataset, which can restrict the model's applicability in real-world scenarios. Additionally, a potential limitation is the risk of discarding important features during PCA, which might reduce the classifier's performance in datasets that have more varied or complex sign language. Future work may focus on expanding the dataset in order to include dynamic gestures and increase the sample diversity, which can improve the applicability of the model. Moreover, researchers can experiment on hybrid methods that combine both traditional machine learning and deep learning algorithms to provide more robust solutions for sign language recognition. Finally, using a wider range of evaluation metrics will help to validate the model's performance more detailed and precisely.

7 CONCLUSION

This paper investigated four different approaches for Sign Language Recognition, on the MNIST dataset. The models with PCA consistently perform better than without PCA, especially for the Polynomial kernel (85.75%). This can be interpreted as PCA's ability to reduce noise, aiding the process of detecting tricky letters. Moreover, RBF kernel proves to behave more stably across different labels, while Polynomial is rather more inconsistent. In conclusion, despite not being able to obtain such high accuracy like Deep learning methods, SVM and PCA combination is no doubt an efficient solution for classification tasks on smaller datasets. With regularization, these models are less prone to overfitting and achieve a sufficiently good accuracy, with higher consistency using RBF Kernel, and more accurate while being less stable using Polynomial Kernel.

REFERENCES

- [1] Pigou, Lionel et al. "Sign Language Recognition Using Convolutional Neural Networks." ECCV Workshops (2014).
- [2] Poladiya, Parth et al. "Sign Language Detection Using Deep Learning." 2024 3rd International Conference for Innovation in Technology (INOCIN) (2024): 1-6.
- [3] Imantoko, Imantoko et al. "Comparative analysis of support vector machine and k-nearest neighbors with a pyramidal histogram of the gradient for sign language detection." Matrix : Jurnal Manajemen Teknologi dan Informatika (2021): n. pag.
- [4] Steinwart, Ingo and Andreas Christmann. "Support vector machines." Wiley Interdisciplinary Reviews: Computational Statistics 1 (2008): n. pag.
- [5] Shen, Heng Tao. "Principal Component Analysis." Technometrics 45 (2003): 276 - 276.
- [6] Novianty, Astri and Fairuz Azmi. "Sign Language Recognition using Principal Component Analysis and Support Vector Machine." IJAIT (International Journal of Applied Information Technology) (2021): n. pag.
- [7] Lwin, Thida and Myint Thu Zar Tun. "Real-time Myanmar Sign Language Recognition System using PCA and SVM." International Journal of Trend in Scientific Research and Development (2019): n. pag.
- [8] Hai, Pham The et al. "Automatic feature extraction for Vietnamese sign language recognition using support vector machine." 2018 2nd International Conference on Recent Advances in Signal Processing, Telecommunications & Computing (SigTelCom) (2018): 146-151.
- [9] Wang, Chengliang et al. "Face Recognition Based on Principle Component Analysis and Support Vector Machine." 2011 3rd International Workshop on Intelligent Systems and Applications (2011): 1-4.
- [10] G, Ramesh et al. "An Efficient Method for Handwritten Kannada Digit Recognition based on PCA and SVM Classifier." Journal of Information Systems and Telecommunication (2021): n. pag.
- [11] Sakr, George E. et al. "Comparing deep learning and support vector machines for autonomous waste sorting." 2016 IEEE International Multidisciplinary Conference on Engineering Technology (IMCET) (2016): 207-212.
- [12] S, Shivashankar and Srinath S. "American Sign Language Recognition System: An Optimal Approach." International Journal of Image, Graphics and Signal Processing (2018): n. pag.
- [13] Valli, Clayton, and Ceil Lucas. "Linguistics of American sign language: An introduction." Gallaudet University Press, 2000.